

Combined Exercise Markdown Files

Generated from `./docs`

- elk_query_practice_5_questions.md
- influxdb_flux_query_practice_5_questions.md
- NEO4J_MODELING_QUESTIONS.md
- NEO4J_QUERY_QUESTIONS.md
- QUERY_QUESTIONS.md

elk_query_practice_5_questions.md

Elasticsearch / ELK Query Practice

This file contains **5 practice questions** and **5 complete answers** covering the most common Elasticsearch Query DSL patterns used in exam-style questions.

In the ELK stack, the actual search engine is **Elasticsearch**. So when we say “ELK queries”, we usually mean **Elasticsearch Query DSL**.

Basic Mapping Used in These Questions

Assume we have this index:

```
products
```

And this mapping:

```
{
  "mappings": {
    "properties": {
      "name": {
        "type": "text"
      },
      "description": {
        "type": "text"
      },
      "category": {
        "type": "keyword"
      },
      "brand": {
        "type": "keyword"
      },
      "tags": {
        "type": "keyword"
      },
      "price": {
        "type": "float"
      },
      "rating": {
        "type": "float"
      },
      "in_stock": {
        "type": "boolean"
      },
      "created_at": {
        "type": "date"
      }
    }
  }
}
```

Quick Cheat Sheet

| Need | Query / Feature |

|---|---|

| Search words in a `text` field | `match` |

| Search exact phrase in a `text` field | `match_phrase` |

| Search same text in many fields | `multi_match` |

| Exact value on `keyword` or `boolean` | `term` |

| Field is one of many exact values | `terms` |

| Numeric/date comparison | `range` |

| Field exists | `exists` |

| Combine conditions | `bool` |

| AND | `must` or `filter` |

| OR | `should` + `minimum_should_match` |

| NOT | `must_not` |

| Projection / returned fields | `_source` |

| Sorting | `sort` |

| Limit results | `size` |

| Pagination | `from` + `size` |

| Grouping | `aggs` with `terms`, `date_histogram`, etc. |

Filter and Group Examples

Filter example

Use `filter` inside a `bool` query when the condition is exact and does not need relevance scoring.

This example returns products that are in stock, in the `electronics` category, and cost at most `900`.

```
GET products/_search
{
  "query": {
    "bool": {
      "filter": [
        {
          "term": {
            "in_stock": true
          }
        }
      ]
    }
  }
}
```

```

    },
    {
      "term": {
        "category": "electronics"
      }
    },
    {
      "range": {
        "price": {
          "lte": 900
        }
      }
    }
  ]
}
}
}

```

SQL-style idea:

```

WHERE in_stock = true
AND category = 'electronics'
AND price <= 900

```

Group example

Use `aggs` for grouping. A `terms` aggregation is the most common way to group by an exact-value field.

This example groups products by `category` and counts how many products are in each category.

```

GET products/_search
{
  "size": 0,
  "aggs": {
    "products_by_category": {
      "terms": {
        "field": "category"
      }
    }
  }
}

```

SQL-style idea:

```

SELECT category, COUNT(*)
FROM products
GROUP BY category;

```

Filter first, then group

Filtering and grouping are often used together. The `query` filters the documents first, then `aggs` groups only the matching documents.

This example filters to in-stock electronics products, then groups them by `brand`.

```
GET products/_search
{
  "size": 0,
  "query": {
    "bool": {
      "filter": [
        {
          "term": {
            "in_stock": true
          }
        },
        {
          "term": {
            "category": "electronics"
          }
        }
      ]
    }
  },
  "aggs": {
    "products_by_brand": {
      "terms": {
        "field": "brand"
      },
      "aggs": {
        "average_price": {
          "avg": {
            "field": "price"
          }
        }
      }
    }
  }
}
```

SQL-style idea:

```
SELECT brand, COUNT(*), AVG(price)
FROM products
WHERE in_stock = true
      AND category = 'electronics'
GROUP BY brand;
```

Question 1 — Basic Search with AND Conditions

Write a query to retrieve products where:

1. `name` contains the word `laptop`
2. `category` is exactly `electronics`
3. `price` is less than or equal to `900`

4. `in_stock` is `true`

Answer 1

```
GET products/_search
{
  "query": {
    "bool": {
      "must": [
        {
          "match": {
            "name": "laptop"
          }
        },
        {
          "term": {
            "category": "electronics"
          }
        },
        {
          "range": {
            "price": {
              "lte": 900
            }
          }
        },
        {
          "term": {
            "in_stock": true
          }
        }
      ]
    }
  }
}
```

Concepts used

- `match` because `name` is a `text` field.
 - `term` because `category` is a `keyword` field.
 - `range` because `price` is numeric.
 - `term` because `in_stock` is boolean.
 - `must` because all conditions must be true.
-

Question 2 — Phrase Search Across Multiple Fields + Projection + Sorting

Write a query to retrieve products where:

1. the exact phrase `wireless headphones` appears in either `name` or `description`

2. `rating` is greater than or equal to `4.0`
 3. return only `name`, `brand`, `price`, and `rating`
 4. return only the first `10` results
 5. sort results by `price` ascending
-

Answer 2

```
GET products/_search
{
  "_source": ["name", "brand", "price", "rating"],
  "size": 10,
  "sort": [
    {
      "price": {
        "order": "asc"
      }
    }
  ],
  "query": {
    "bool": {
      "must": [
        {
          "multi_match": {
            "query": "wireless headphones",
            "fields": ["name", "description"],
            "type": "phrase"
          }
        },
        {
          "range": {
            "rating": {
              "gte": 4.0
            }
          }
        }
      ]
    }
  }
}
```

Concepts used

- `multi_match` searches the same query in multiple fields.
 - `type: "phrase"` means exact phrase search.
 - `_source` is projection: it controls which fields are returned.
 - `size` limits the number of returned results.
 - `sort` orders the results.
-

Question 3 — OR, NOT, and Exists

Write a query to retrieve products where:

1. `category` is exactly `electronics` OR `gaming`
 2. `description` exists
 3. `brand` is NOT `Unknown`
-

Answer 3

```
GET products/_search
{
  "query": {
    "bool": {
      "filter": [
        {
          "bool": {
            "should": [
              {
                "term": {
                  "category": "electronics"
                }
              },
              {
                "term": {
                  "category": "gaming"
                }
              }
            ],
            "minimum_should_match": 1
          }
        },
        {
          "exists": {
            "field": "description"
          }
        }
      ],
      "must_not": [
        {
          "term": {
            "brand": "Unknown"
          }
        }
      ]
    }
  }
}
```

Concepts used

- `should` represents OR.
- `minimum_should_match: 1` means at least one OR condition must match.
- `exists` checks whether the field is present.
- `must_not` represents NOT.
- `filter` is often used for exact conditions that do not need scoring.

Shorter alternative for the OR condition

Because both OR conditions are on the same field, we can also use `terms` :

```
{
  "terms": {
    "category": ["electronics", "gaming"]
  }
}
```

Question 4 — Grouping with Aggregations

Write a query to answer this question:

For products that are in stock, group them by `category` and calculate the average, minimum, and maximum `price` for each category.

Do not return actual product documents, only the aggregation result.

Answer 4

```
GET products/_search
{
  "size": 0,
  "query": {
    "term": {
      "in_stock": true
    }
  },
  "aggs": {
    "products_by_category": {
      "terms": {
        "field": "category"
      },
      "aggs": {
        "average_price": {
          "avg": {
            "field": "price"
          }
        },
        "minimum_price": {
          "min": {
            "field": "price"
          }
        },
        "maximum_price": {
          "max": {
            "field": "price"
          }
        }
      }
    }
  }
}
```

```
}  
}
```

Concepts used

- `size: 0` means do not return documents, only aggregation results.
- `aggs` is used for grouping/statistics.
- `terms` aggregation groups by exact values, similar to SQL `GROUP BY`.
- `avg`, `min`, and `max` are metric aggregations.
- `doc_count` in the result is similar to `COUNT(*)`.

Question 5 — Date Range + Date Histogram + Pagination

Write a query to retrieve products where:

1. `created_at` is from `2026-01-01` onward
2. `rating` is greater than or equal to `4.0`
3. return only `name`, `category`, `rating`, and `created_at`
4. return page 2, assuming each page has 10 results
5. sort newest products first
6. also group matching products by month using `created_at`

Answer 5

```
GET products/_search  
{  
  "_source": ["name", "category", "rating", "created_at"],  
  "from": 10,  
  "size": 10,  
  "sort": [  
    {  
      "created_at": {  
        "order": "desc"  
      }  
    }  
  ],  
  "query": {  
    "bool": {  
      "filter": [  
        {  
          "range": {  
            "created_at": {  
              "gte": "2026-01-01"  
            }  
          }  
        }  
      ]  
    }  
  }  
}
```

```

    }
  },
  {
    "range": {
      "rating": {
        "gte": 4.0
      }
    }
  }
]
}
},
"aggs": {
  "products_per_month": {
    "date_histogram": {
      "field": "created_at",
      "calendar_interval": "month"
    }
  }
}
}
}

```

Concepts used

- Date comparisons also use `range`.
- Date values are strings, so they use quotes.
- `from: 10` and `size: 10` means page 2 if page 1 starts at `from: 0`.
- `_source` is projection.
- `sort` orders by date descending.
- `date_histogram` groups documents by time intervals.

Final Syntax Reminders

Quotes

Keys	-> quoted
String values	-> quoted
Numbers	-> not quoted
Booleans true/false	-> not quoted
null	-> not quoted
Arrays []	-> not quoted as a whole
Objects {}	-> not quoted as a whole

Examples:

```

{
  "term": {
    "category": "electronics"
  }
}

```

```
{
  "range": {
    "price": {
      "lte": 900
    }
  }
}
```

```
{
  "term": {
    "in_stock": true
  }
}
```

Main Template to Memorize

```
GET index_name/_search
{
  "query": {
    "bool": {
      "must": [
        {
          "QUERY_TYPE": {
            "field": "value"
          }
        }
      ],
      "filter": [
        {
          "QUERY_TYPE": {
            "field": "value"
          }
        }
      ],
      "should": [
        {
          "QUERY_TYPE": {
            "field": "value"
          }
        }
      ],
      "must_not": [
        {
          "QUERY_TYPE": {
            "field": "value"
          }
        }
      ]
    }
  }
}
```

influxdb_flux_query_practice_5_questions.md

InfluxDB Flux Query Practice

This file contains **5 practice questions** and **5 complete answers** covering the most common InfluxDB **Flux** query patterns used in exam-style questions.

The focus is on fast learning: each question has a complete query, explanation, and common syntax notes.

Basic Data Model Used in These Questions

Assume the data is stored in InfluxDB with:

```
bucket: IoT
measurement: home
fields: temp, hum
tag: room
```

Example data conceptually looks like this:

```
| _time | _measurement | room | _field | _value |
|---|---|---|---|---|
| 2026-06-21T10:00:00Z | home | kitchen | temp | 24.5 |
| 2026-06-21T10:00:00Z | home | kitchen | hum | 55 |
| 2026-06-21T10:00:00Z | home | bedroom | temp | 22.1 |
| 2026-06-21T10:00:00Z | home | bedroom | hum | 60 |
```

Flux Mental Model

Most Flux queries follow this pipeline:

```
from(bucket: "bucket_name")
  |> range(start: -time)
  |> filter(fn: (r) => condition)
  |> transformation()
```

Important built-in columns:

```
| Column | Meaning |
|---|---|
| _time | timestamp of the point |
```

| `_measurement` | measurement name, like a table name |

| `_field` | field name, for example `temp` or `hum` |

| `_value` | actual numeric/string value of the field |

| `room` | tag column in this example |

Quick Cheat Sheet

| Need | Flux function / syntax |

|---|---|

| Select bucket | `from(bucket: "IoT")` |

| Select time range | `range(start: -2h)` |

| Filter measurement | `filter(fn: (r) => r._measurement == "home")` |

| Filter field | `filter(fn: (r) => r._field == "temp")` |

| Filter tag | `filter(fn: (r) => r.room == "kitchen")` |

| Filter value | `filter(fn: (r) => r._value > 25.0)` |

| Average | `mean()` |

| Minimum | `min()` |

| Maximum | `max()` |

| Count | `count()` |

| Latest value | `last()` |

| Group by tag | `group(columns: ["room"])` |

| Windowed aggregation | `aggregateWindow(every: 10m, fn: mean)` |

| Sort | `sort(columns: ["_time"], desc: true)` |

| Limit rows | `limit(n: 10)` |

| Select columns | `keep(columns: ["_time", "room", "_value"])` |

| Convert fields into columns | `pivot(...)` |

Question 1 — Average Temperature Per Room Over the Last 2 Hours

Write a Flux query to compute the **average temperature per room** over the last **2 hours**.

Use:

```
bucket: IoT
measurement: home
field: temp
tag: room
```

Answer 1

```
from(bucket: "IoT")
  |> range(start: -2h)
  |> filter(fn: (r) => r._measurement == "home")
  |> filter(fn: (r) => r._field == "temp")
  |> group(columns: ["room"])
  |> mean()
```

Explanation

```
from(bucket: "IoT")
```

Selects the bucket.

```
range(start: -2h)
```

Keeps only data from the last 2 hours.

```
filter(fn: (r) => r._measurement == "home")
```

Keeps only rows from the `home` measurement.

```
filter(fn: (r) => r._field == "temp")
```

Keeps only temperature values.

```
group(columns: ["room"])
```

Groups data by room.

```
mean()
```

Computes the average `_value` for each room.

SQL-like idea

```
SELECT room, AVG(temp)
FROM home
```

```
WHERE time >= now() - 2 hours
GROUP BY room;
```

Question 2 — Average Temperature Every 10 Minutes Per Room

Write a Flux query to compute the **average temperature every 10 minutes per room** over the last **2 hours**.

This is different from Question 1 because now we want a time series of averages, not one average per room.

Answer 2

```
from(bucket: "IoT")
  |> range(start: -2h)
  |> filter(fn: (r) => r._measurement == "home")
  |> filter(fn: (r) => r._field == "temp")
  |> group(columns: ["room"])
  |> aggregateWindow(every: 10m, fn: mean, createEmpty: false)
```

Explanation

```
group(columns: ["room"])
```

Keeps each room separate.

```
aggregateWindow(every: 10m, fn: mean, createEmpty: false)
```

Divides time into 10-minute windows and calculates the average temperature in each window.

`createEmpty: false` avoids returning empty time windows with no data.

When to use `mean()` vs `aggregateWindow()`

Use this:

```
mean()
```

when you want **one final average**.

Use this:

```
aggregateWindow(every: 10m, fn: mean)
```

when you want **averages over repeated time windows**.

Question 3 — Latest Humidity Value in One Room

Write a Flux query to retrieve the **latest humidity value** measured in the `kitchen` over the last **24 hours**.

Return only:

```
_time, room, _field, _value
```

Answer 3

```
from(bucket: "IoT")
  > range(start: -24h)
  > filter(fn: (r) => r._measurement == "home")
  > filter(fn: (r) => r._field == "hum")
  > filter(fn: (r) => r.room == "kitchen")
  > last()
  > keep(columns: ["_time", "room", "_field", "_value"])
```

Explanation

```
filter(fn: (r) => r._field == "hum")
```

Selects humidity readings.

```
filter(fn: (r) => r.room == "kitchen")
```

Selects only the kitchen. `room` is a tag.

```
last()
```

Returns the latest point in the selected time range.

```
keep(columns: ["_time", "room", "_field", "_value"])
```

Acts like projection: it keeps only the requested columns.

SQL-like idea

```
SELECT time, room, hum
FROM home
WHERE room = 'kitchen'
```

```
ORDER BY time DESC
LIMIT 1;
```

Question 4 — Retrieve Temperature and Humidity Together Using Pivot

Write a Flux query to retrieve both `temp` and `hum` for the `bedroom` over the last **1 hour**, and convert them into separate columns.

The output should look conceptually like:

```
|_time | room | temp | hum |
|---|---|---|---|
| 10:00 | bedroom | 22.4 | 58 |
| 10:05 | bedroom | 22.5 | 57 |
```

Answer 4

```
from(bucket: "IoT")
  |> range(start: -1h)
  |> filter(fn: (r) => r._measurement == "home")
  |> filter(fn: (r) => r.room == "bedroom")
  |> filter(fn: (r) => r._field == "temp" or r._field == "hum")
  |> pivot(rowKey: ["_time"], columnKey: ["_field"], valueColumn: "_value")
  |> keep(columns: ["_time", "room", "temp", "hum"])
```

Explanation

In InfluxDB, fields are usually stored vertically:

```
|_time | _field | _value |
|---|---|---|
| 10:00 | temp | 22.4 |
| 10:00 | hum | 58 |
```

`pivot()` converts this into a wider table:

```
|_time | temp | hum |
|---|---|---|
| 10:00 | 22.4 | 58 |
```

The key part is:

```
pivot(rowKey: ["_time"], columnKey: ["_field"], valueColumn: "_value")
```

Meaning:

```
Each time becomes a row.  
Each field name becomes a column.  
The value comes from _value.
```

Question 5 — Count High Temperature Readings Per Room

Write a Flux query to count how many temperature readings were greater than `28.0` in each room over the last **24 hours**.

Use:

```
measurement: home  
field: temp  
threshold: _value > 28.0  
grouping tag: room
```

Answer 5

```
from(bucket: "IoT")  
  |> range(start: -24h)  
  |> filter(fn: (r) => r._measurement == "home")  
  |> filter(fn: (r) => r._field == "temp")  
  |> filter(fn: (r) => r._value > 28.0)  
  |> group(columns: ["room"])  
  |> count()
```

Explanation

```
filter(fn: (r) => r._value > 28.0)
```

Keeps only temperature readings above 28.0.

```
group(columns: ["room"])
```

Separates the data by room.

```
count()
```

Counts how many rows are left in each room group.

SQL-like idea

```
SELECT room, COUNT(*)  
FROM home  
WHERE temp > 28.0  
AND time >= now() - 24 hours  
GROUP BY room;
```

Common Flux Syntax Rules

1. Strings use quotes

```
r._measurement == "home"  
r._field == "temp"  
r.room == "kitchen"
```

2. Durations do not use quotes

Correct:

```
range(start: -2h)
```

Wrong:

```
range(start: "-2h")
```

3. Numbers do not use quotes

Correct:

```
r._value > 28.0
```

Wrong:

```
r._value > "28.0"
```

4. Booleans do not use quotes

```
true  
false
```

5. Every pipeline step uses `|>`

```
from(bucket: "IoT")
  |> range(start: -2h)
  |> filter(fn: (r) => r._field == "temp")
  |> mean()
```

6. Filter functions usually look like this

```
filter(fn: (r) => r.columnName == "value")
```

Examples:

```
filter(fn: (r) => r._measurement == "home")
filter(fn: (r) => r._field == "temp")
filter(fn: (r) => r.room == "kitchen")
filter(fn: (r) => r._value > 25.0)
```

Most Important Template to Memorize

```
from(bucket: "BUCKET_NAME")
  |> range(start: -TIME_RANGE)
  |> filter(fn: (r) => r._measurement == "MEASUREMENT_NAME")
  |> filter(fn: (r) => r._field == "FIELD_NAME")
  |> filter(fn: (r) => r.TAG_NAME == "TAG_VALUE")
  |> group(columns: ["TAG_NAME"])
  |> AGGREGATION_FUNCTION()
```

Example:

```
from(bucket: "IoT")
  |> range(start: -2h)
  |> filter(fn: (r) => r._measurement == "home")
  |> filter(fn: (r) => r._field == "temp")
  |> group(columns: ["room"])
  |> mean()
```

Final Exam Tips

1. Always start with `from(bucket: "...")`.
2. Always add a `range(...)`; Flux usually requires a time range for queries.
3. Filter `_measurement` to select the measurement.
4. Filter `_field` to select the field.
5. Filter tags directly, like `r.room == "kitchen"`.

6. Use `_value` for numeric comparisons.
7. Use `group(columns: [...])` before aggregation if the question asks "per room", "per device", or "per host".
8. Use `aggregateWindow(...)` when the question says "every 5 minutes", "per hour", "daily average", etc.
9. Use `pivot(...)` when the question wants multiple fields as columns.
10. Use `keep(...)` when the question asks to return only specific columns.

Neo4j Graph Modeling Questions

These exercises focus on designing a Neo4j graph schema from requirements.

For each case, identify:

- node labels and properties
- relationship types and directions
- relationship properties if needed

Question 1 — Employee training platform

A company runs an internal training platform where employees can enroll in courses.

The platform must support these queries efficiently:

- Which courses has a given employee completed, and what score did they get?
- Which employees are currently enrolled in a given course?
- Which instructor teaches a given course?
- Which employees share at least one completed course?

Requirements:

- Each employee has a unique identifier, name, and department
- Each course has a unique identifier, title, duration in hours, and difficulty level
- Each instructor has a name and an expertise field
- Track the date when an employee enrolls in a course
- Track the date and score when an employee completes a course
- A course is taught by one instructor

Design the Neo4j graph schema for this platform.

Question 2 — Music streaming platform

A music streaming service wants to model listeners, songs, and playlists.

The platform must support these queries efficiently:

- Which songs are in a given playlist?
- Which playlists contain a given song?
- Which listeners follow a given artist?
- Which listeners have liked the same songs?

Requirements:

- Each listener has a unique id, username, and country
- Each artist has a unique id, name, and genre

- Each song has a unique id, title, duration, and release year
- Each playlist has a unique id, title, and privacy status
- Track when a listener adds a song to a playlist
- Track when a listener likes a song

Design the Neo4j graph schema for this platform.

Question 3 — Hospital appointment system

A hospital wants to manage patients, doctors, appointments, and prescriptions.

The platform must support these queries efficiently:

- Which appointments does a given patient have?
- Which doctor is assigned to a given appointment?
- Which prescriptions were written during a given appointment?
- Which patients have visited the same doctor?

Requirements:

- Each patient has a unique id, name, age, and contact information
- Each doctor has a unique id, name, and specialty
- Each appointment has a unique id, date, time, and status
- Each prescription has a unique id, medication name, dosage, and duration
- Track the status of the appointment
- Track the prescription date

Design the Neo4j graph schema for this platform.

Question 4 — Online learning community

An online learning community wants to model students, posts, comments, and tags.

The platform must support these queries efficiently:

- Which posts did a given student create?
- Which comments belong to a given post?
- Which students reacted to the same post?
- Which posts share at least one tag?

Requirements:

- Each student has a unique id, name, and city
- Each post has a unique id, title, body, and creation time
- Each comment has a unique id, text, and creation time
- Each tag is identified by a name
- Track the type of reaction a student gives to a post
- A comment belongs to exactly one post

Design the Neo4j graph schema for this platform.

Question 5 — Food delivery application

A food delivery application wants to manage customers, restaurants, orders, and menu items.

The platform must support these queries efficiently:

- Which restaurants does a customer order from most often?
- Which menu items belong to a given restaurant?
- Which orders contain a given menu item?
- Which customers have ordered from the same restaurant?

Requirements:

- Each customer has a unique id, name, and city
- Each restaurant has a unique id, name, category, and rating
- Each order has a unique id, order date, status, and total amount
- Each menu item has a unique id, name, price, and availability
- Track the quantity of each item in an order
- A restaurant offers many menu items

Design the Neo4j graph schema for this platform.

NEO4J_QUERY_QUESTIONS.md

Neo4j Query Questions

These questions are based on the social network analytics graph:

- (:User)
- (:Content)
- (:Venue)
- [:FOLLOWS]
- [:LIKED]
- [:SIMILAR_TO]

Question 1 — User follow network

Given that users are connected through `FOLLOWS` relationships:

- write a Cypher query that returns the users followed by a given user
- include the followed user id and the relationship creation time
- sort results by the follow time in descending order

Answer

```
MATCH (u:User {id: "user_001"})-[f:FOLLOWS]->(u2:User)
RETURN u2.id AS followed_user_id, u2.username AS followed_username, f.created_at AS follow_time
ORDER BY follow_time DESC
```

Question 2 — Most liked posts

Given that users like content through `LIKED` relationships:

- write a Cypher query that finds the top 10 posts with the highest number of incoming `LIKED` relationships
- return the post id and number of likes
- sort by likes in descending order

Answer

```
MATCH (:User)-[:LIKED]->(p:Post)
WITH p, count(*) AS total_likes
RETURN p.id AS post_id, total_likes
ORDER BY total_likes DESC
LIMIT 10
```

Question 3 — Similar content recommendations

Given that some posts are connected with `SIMILAR_TO`:

- write a Cypher query that finds posts similar to a given post
- return the similar post id, similarity score, and shared hashtags if available
- only keep results with a similarity score greater than or equal to 0.5

Answer

```
MATCH (p:Post {id: "content_2050"})<-[:SIMILAR_TO]-(p2:Post)
WHERE r.score >= 0.5
RETURN p2.id AS similar_post_id, r.score AS similarity_score, r.shared_hashtags AS shared_hashtags
ORDER BY similarity_score DESC
```

Question 4 — Taste-based user recommendation

Given a user and the content they have liked:

- write a Cypher query that finds other users who liked the same content
- count how many liked posts they have in common
- return the user ids and the overlap count
- sort by overlap count in descending order

Answer

```
MATCH (u1:User {id: "user_004"})-[:LIKED]->(c:Content)<-[:LIKED]-(u2:User)
WHERE u1 <> u2
WITH u1.id AS main_user, u2.id AS similar_user, count(DISTINCT c) AS overlap_count
RETURN main_user, similar_user, overlap_count
ORDER BY overlap_count DESC
LIMIT 10
```

Question 5 — Venue popularity

Given venue nodes in the graph:

- write a Cypher query that finds venues followed by the most users
- return the venue id and the number of followers
- sort by follower count in descending order

Answer

```
MATCH (:User)-[:FOLLOWS]->(v:Venue)
WITH v, count(*) AS follower_count
RETURN v.id AS venue_id, v.name AS venue_name, follower_count
ORDER BY follower_count DESC
LIMIT 5
```

QUERY_QUESTIONS.md

Query Questions

These questions are based on the social network analytics database:

- `users`
- `content`
- `interactions`
- `relationships`
- `venues`

MongoDB Filter and Group Examples

Simple filter with `find`

Use `find` when you only need to filter and return matching documents.

This example returns food posts with at least `20` likes and only shows selected fields.

```
db.content.find(
  {
    type: "post",
    style: "food",
    "metrics.like_count": { $gte: 20 }
  },
  {
    _id: 1,
    author_id: 1,
    style: 1,
    hashtags: 1,
    "metrics.like_count": 1
  }
).sort({
  "metrics.like_count": -1
}).limit(10)
```

SQL-style idea:

```
SELECT _id, author_id, style, hashtags, metrics.like_count
FROM content
WHERE type = 'post'
      AND style = 'food'
      AND metrics.like_count >= 20
ORDER BY metrics.like_count DESC
LIMIT 10;
```

More `find` query examples

Use `find` for normal document retrieval. It can filter, project fields, sort, skip, and limit results.

Important: `find` does not group documents. For grouping, use `aggregate` with `$group`.

Exact match

Find users from Turin.

```
db.users.find(
  {
    city: "Turin"
  },
  {
    _id: 0,
    username: 1,
    display_name: 1,
    city: 1
  }
)
```

Comparison filter

Find venues with rating at least 4.5 and at least 200 votes.

```
db.venues.find(
  {
    "rating.aggregate_rating": { $gte: 4.5 },
    "rating.votes": { $gte: 200 }
  },
  {
    _id: 0,
    name: 1,
    city: 1,
    category: 1,
    "rating.aggregate_rating": 1,
    "rating.votes": 1
  }
).sort({
  "rating.votes": -1
})
```

Array contains value

Find content that contains the hashtag culture .

```
db.content.find(
  {
    hashtags: "culture"
  },
  {
    _id: 1,
    author_id: 1,
    style: 1,
    hashtags: 1
  }
)
```

Match one of several values with \$in

Find posts whose style is either food , travel , or culture .

```
db.content.find(
  {
    type: "post",
    style: {
      $in: ["food", "travel", "culture"]
    }
  },
  {
    _id: 1,
    author_id: 1,
    style: 1,
    category: 1
  }
)
```

OR condition with \$or

Find venues that are either highly rated or have many votes.

```
db.venues.find(
  {
    $or: [
      {
        "rating.aggregate_rating": { $gte: 4.8 }
      },
      {
        "rating.votes": { $gte: 500 }
      }
    ]
  },
  {
    _id: 0,
    name: 1,
    city: 1,
    "rating.aggregate_rating": 1,
    "rating.votes": 1
  }
)
```

Array size or existence

Find users who have at least one recent interaction.

```
db.users.find(
  {
    "recent_interactions.0": {
      $exists: true
    }
  },
  {
    _id: 0,
    username: 1,
    city: 1,
    recent_interactions: 1
  }
)
```

Date range

Find relationships created on or after `2026-06-01`.

```
db.relationships.find(
  {
    created_at: {
      $gte: new Date("2026-06-01T00:00:00Z")
    }
  },
  {
    _id: 1,
    type: 1,
    source_user_id: 1,
    target_type: 1,
    target_id: 1,
    created_at: 1
  }
).sort({
  created_at: -1
})
```

Pagination with `skip` and `limit`

Find the second page of food posts, assuming `10` results per page.

```
db.content.find(
  {
    type: "post",
    style: "food"
  },
  {
    _id: 1,
    author_id: 1,
    style: 1,
    "metrics.like_count": 1
  }
).sort({
  "metrics.like_count": -1
}).skip(10).limit(10)
```

Filter with `$match` in an aggregation pipeline

Use `$match` when filtering is part of a larger aggregation pipeline.

This example filters venues by city, rating, and vote count.

```
db.venues.aggregate([
  {
    $match: {
      city: "Makati City",
      "rating.aggregate_rating": { $gte: 4.5 },
      "rating.votes": { $gte: 200 }
    }
  },
  {
    $project: {
      _id: 0,
      name: 1,
      city: 1,
      category: 1,

```

```

    "rating.aggregate_rating": 1,
    "rating.votes": 1
  }
}
])

```

SQL-style idea:

```

SELECT name, city, category, rating.aggregate_rating, rating.votes
FROM venues
WHERE city = 'Makati City'
      AND rating.aggregate_rating >= 4.5
      AND rating.votes >= 200;

```

Group with \$group

Use `$group` to group documents and calculate aggregate values such as counts, sums, and averages.

This example groups content by `style` and calculates the number of posts plus average likes.

```

db.content.aggregate([
  {
    $match: {
      type: "post"
    }
  },
  {
    $group: {
      _id: "$style",
      total_posts: { $sum: 1 },
      average_likes: { $avg: "$metrics.like_count" }
    }
  },
  {
    $sort: {
      total_posts: -1
    }
  }
])

```

SQL-style idea:

```

SELECT style, COUNT(*) AS total_posts, AVG(metrics.like_count) AS average_likes
FROM content
WHERE type = 'post'
GROUP BY style
ORDER BY total_posts DESC;

```

Filter first, then group

In MongoDB aggregation, `$match` normally comes before `$group`. This filters the input documents first, then groups only the matching documents.

This example filters recent `like` relationships for content, then groups them by user.

```

db.relationships.aggregate([
  {
    $match: {
      type: "like",
      target_type: "content",
      created_at: {
        $gte: new Date("2026-06-01T00:00:00Z")
      }
    }
  },
  {
    $group: {
      _id: "$source_user_id",
      total_likes: { $sum: 1 }
    }
  },
  {
    $project: {
      _id: 0,
      source_user_id: "$_id",
      total_likes: 1
    }
  },
  {
    $sort: {
      total_likes: -1
    }
  }
])

```

SQL-style idea:

```

SELECT source_user_id, COUNT(*) AS total_likes
FROM relationships
WHERE type = 'like'
      AND target_type = 'content'
      AND created_at >= '2026-06-01T00:00:00Z'
GROUP BY source_user_id
ORDER BY total_likes DESC;

```

Question 1 — User activity profile

Given the `users` collection:

```

{
  "_id": "user_001",
  "username": "user_001",
  "display_name": "User 0001",
  "city": "Turin",
  "interests": ["pasta", "museums", "street_food"],
  "recent_interactions": [
    {
      "type": "view",
      "content_id": "content_010",
      "created_at": "2026-06-10T12:30:00Z"
    }
  ]
}

```

Write a MongoDB query that retrieves all users from the city "Turin" who have at least one recent interaction, and returns only:

- username
- display_name
- city
- the number of recent interactions

Sort the results by number of recent interactions in descending order.

Answer

```
db.users.aggregate([
  {
    $match: {
      city: "Turin",
      "recent_interactions.0": { $exists: true }
    }
  },
  {
    $project: {
      _id: 0,
      username: 1,
      display_name: 1,
      city: 1,
      recent_interactions_count: { $size: "$recent_interactions" }
    }
  },
  {
    $sort: { recent_interactions_count: -1 }
  }
])
```

Question 2 — Popular content by style

Given the content collection:

```
{
  "_id": "content_120",
  "type": "post",
  "author_id": "user_014",
  "style": "food",
  "category": "food",
  "hashtags": ["pasta", "local", "restaurant"],
  "metrics": {
    "view_count": 123,
    "like_count": 41,
    "comment_count": 8
  }
}
```

Write a MongoDB query that finds the top 10 post documents with:

- style = "food"
- at least 20 likes

- at least 50 views

Return only:

- `_id`
- `author_id`
- `style`
- `hashtags`
- `metrics.view_count`
- `metrics.like_count`

Sort by `metrics.like_count` in descending order.

Answer

```
db.content.aggregate([
  {
    $match: {
      type: "post",
      style: "food",
      "metrics.like_count": { $gte: 20 },
      "metrics.view_count": { $gte: 50 }
    }
  },
  {
    $sort: { "metrics.like_count": -1 }
  },
  {
    $limit: 10
  },
  {
    $project: {
      _id: 1,
      author_id: 1,
      style: 1,
      hashtags: 1,
      "metrics.view_count": 1,
      "metrics.like_count": 1
    }
  }
])
```

Question 3 — Venue engagement

Given the `venues` collection:

```
{
  "_id": "restaurant_6317637",
  "name": "Le Petit Souffle",
  "city": "Makati City",
  "category": "French",
  "cuisines": ["French", "Japanese", "Desserts"],
  "rating": {
    "aggregate_rating": 4.8,
    "votes": 314
  }
}
```

```
}  
}
```

Write a MongoDB query that retrieves venues that:

- belong to city "Makati City"
- have an aggregate rating greater than or equal to 4.5
- have at least 200 votes

Return only:

- name
- city
- category
- cuisines
- rating.aggregate_rating
- rating.votes

Sort by rating.votes in descending order.

Answer

```
db.venues.aggregate([  
  {  
    $match: {  
      city: "Makati City",  
      "rating.aggregate_rating": { $gte: 4.5 },  
      "rating.votes": { $gte: 200 }  
    }  
  },  
  {  
    $project: {  
      _id: 0,  
      name: 1,  
      city: 1,  
      category: 1,  
      cuisines: 1,  
      "rating.aggregate_rating": 1,  
      "rating.votes": 1  
    }  
  },  
  {  
    $sort: { "rating.votes": -1 }  
  }  
])
```

Question 4 — Relationship analysis

Given the relationships collection:

```
{  
  "_id": "relationship_001",  
  "type": "like",  
  "source_user_id": "user_003",  
  "target_type": "content",  
}
```

```
"target_id": "content_045",
"created_at": "2026-05-12T09:00:00Z"
}
```

Write a MongoDB query that finds the users who created the most `like` relationships toward content in the last 30 days.

The result must include:

- `source_user_id`
- the number of likes created

Sort the users by number of likes in descending order and return only the top 5.

Answer

```
db.relationships.aggregate([
  {
    $match: {
      type: "like",
      target_type: "content",
      created_at: {
        $gte: new Date(Date.now() - 30 * 24 * 60 * 60 * 1000)
      }
    }
  },
  {
    $group: {
      _id: "$source_user_id",
      total_likes: { $sum: 1 }
    }
  },
  {
    $sort: {
      total_likes: -1
    }
  },
  {
    $limit: 5
  },
  {
    $project: {
      _id: 0,
      source_user_id: "$_id",
      total_likes: 1
    }
  }
])
```

Question 5 — Content overlap in MongoDB

Given the `content` collection:

```
{
  "_id": "content_245",
  "type": "post",
  "author_id": "user_020",
  "style": "travel",
}
```

```
"category": "travel",
"hashtags": ["city", "museum", "culture", "walk"],
"metrics": {
  "view_count": 88,
  "like_count": 17,
  "comment_count": 4
}
}
```

Write a MongoDB query that finds all `post` documents that:

- have `style = "travel"`
- contain the hashtag `"culture"`
- have at least `10` likes

Return only:

- `_id`
- `author_id`
- `style`
- `category`
- `hashtags`
- `metrics.like_count`

Sort the results by `metrics.like_count` in descending order and return only the top 8 documents.

Answer

```
db.content.aggregate([
  {
    $match: {
      type: "post",
      style: "travel",
      hashtags: "culture",
      "metrics.like_count": { $gte: 10 }
    }
  },
  {
    $sort: { "metrics.like_count": -1 }
  },
  {
    $limit: 8
  },
  {
    $project: {
      _id: 1,
      author_id: 1,
      style: 1,
      category: 1,
      hashtags: 1,
      "metrics.like_count": 1
    }
  }
])
```